Module 7 Lab: Session 2: Throughput Under Production Load

Module	M7: Advanced Web API Development
Session	2 of 4
Exercises	3 (HybridCache with observable stampede protection), 4 (Tier-aware rate limiting)

Welcome to Monday Morning

The dashboard goes live Sunday night. By 09:14 Monday the on-call channel is on fire.

200 students log into the TMS. Each opens their dashboard. Each dashboard fetches the “Popular Courses” list once. The “Popular Courses” endpoint queries `Courses` join `Enrollments` and aggregates count per course, 200 ms on a warm cache, 1.4 seconds on a cold cache. The cache was empty when the deploy went out, so the first 50 requests all fired the same query at the same time. Database CPU climbed to 95 %, response times for *every* endpoint passed 6 seconds, and the admin dashboard timed out at 30, including the dashboard ops uses to see why the API is slow.

While that was unfolding, the integration team’s nightly export script, left running with a typo’d retry loop, started hammering `/api/v2/courses/search` at 100 requests per second. The script will run until someone notices, in about 90 minutes.

Both problems are entirely preventable. Session 2 is where you prevent them.

By the end of today, 50 concurrent students hitting an empty cache produces exactly **one** database query (Exercise 3), and one misconfigured script can no longer drown everyone else (Exercise 4). You will *prove* both, in logs, not on faith.

Before You Begin: Session 2 sync

Confirm your Session 1 work is intact:

```
dotnet test     # CQRS tests still green
```

Primary check: Open **Scalar** starting from your API’s HTTPS base URL (**`https://localhost:5001`** in most lab profiles). Try **`.../scalar/v1`** first, then **`.../scalar`** if the first path 404s. Use **Try it** on **`GET /api/v1/courses`** and **`GET /api/v2/courses`** confirm V1 shows deprecation-related headers where your middleware emits them and V2 does not.

Optional scripted replay:

```
curl -i https://localhost:5001/api/v1/courses
curl -i https://localhost:5001/api/v2/courses
```

If any of those fail, fix Session 1 before continuing, Session 2's caching attaches to the V2 read path for courses from Session 1, and Session 2's rate limiting needs the `IExceptionHandler` plumbing for `ProblemDetails`.

Install the Session 2 package:

```
dotnet add TmsApi.Api package Microsoft.Extensions.Caching.Hybrid
```

(Rate limiting ships with `Microsoft.AspNetCore.RateLimiting` in the runtime. No extra NuGet.)

Exercise 3: HybridCache with Observable Stampede Protection (LO 7.2)

Scenario: The "Popular Courses" endpoint is queried 1,000 times per minute on Monday mornings. Every request hits the database with the same query. The dashboard team also asked for one specific guarantee: "tell us, in the logs, how many of those requests actually touched the database." Without that signal you cannot prove the cache works in production. You will implement HybridCache with atomic refetching, tag-based invalidation, **and** a hit/miss log line per call so the cache becomes observable, not magical.

Step 1 Register HybridCache (and decide about Redis later)

Open `TmsApi.Api/Program.cs` and add:

```
builder.Services.AddHybridCache(options =>
{
    options.DefaultEntryOptions = new HybridCacheEntryOptions
    {
        Expiration = TimeSpan.FromMinutes(10),
        LocalCacheExpiration = TimeSpan.FromMinutes(2)
    };
});
```

For this lab, leave the L2 distributed cache as the default in-memory fallback single-node is fine while you build the pattern. **In production**, register a Redis-backed `IDistributedCache` *before* `AddHybridCache` and HybridCache will pick it up automatically:

```
// Production-only  leave commented in lab
// builder.Services.AddStackExchangeRedisCache(options =>
// {
```

```
//     options.Configuration =
builder.Configuration.GetConnectionString("Redis");
//     options.InstanceName = "tms:";
// });
// builder.Services.AddHybridCache();
```

> The instance prefix (`"tms:"`) is the small detail every team forgets. Without it, two services sharing the same Redis cluster collide on `course:CSE-101`. With it, they live in separate keyspaces (`tms:course:CSE-101` vs `billing:course:CSE-101`) and ops can scan a single service's keys with `SCAN MATCH tms:*`.

Step 2 Pick a cache key strategy that survives schema changes

Cache keys are part of your contract with future-you. The first time you change a DTO shape and forget to clear the cache, every server in production will serve stale shapes for ten minutes. The fix is to embed a **schema version** in the key:

```
// In TmsApi.Infrastructure/Caching/CacheKeys.cs
namespace TmsApi.Infrastructure.Caching;

public static class CacheKeys
{
    private const string SchemaVersion = "v2";

    public static string Course(string code) =>
$"{SchemaVersion}:course:{code}";
    public static string CoursesAll => $"{SchemaVersion}:courses:all";
    public const string CoursesTag = "courses";
}
```

When you change `CourseDto`, bump `SchemaVersion` to `"v3"` and old entries become unreachable instantly. No coordinated cache flush, no support page, no incident.

Step 3 Create a cached course service with hit/miss observability

Create `TmsApi.Infrastructure/Services/CachedCourseService.cs`:

```
using Microsoft.Extensions.Caching.Hybrid;
using Microsoft.Extensions.Logging;
using TmsApi.Application.DTOs;
using TmsApi.Application.Interfaces;
using TmsApi.Infrastructure.Caching;

namespace TmsApi.Infrastructure.Services;

public class CachedCourseService(
    HybridCache cache,
    ICourseRepository repo,
    ILogger<CachedCourseService> logger)
```

```
    : ICachedCourseService
{
    public async Task<CourseDto> GetCourseAsync(string code,
CancellationToken ct)
    {
        var key = CacheKeys.Course(code);
        var dbHit = false;

        var dto = await cache.GetOrCreateAsync(
            key,
            (repo, code),
            async (state, token) =>
            {
                dbHit = true;
                logger.LogInformation("Cache MISS for {Key}  fetching from
DB", key);
                var course = await state.repo.GetByCodeAsync(state.code,
token)
                    ?? throw new NotFoundException($"Course {state.code} not
found.");
                return new CourseDto(
                    course.Id, course.Title, course.Code,
                    course.MaxCapacity, course.Enrollments.Count);
            },
            tags: [CacheKeys.CoursesTag],
            cancellationToken: ct);

        if (!dbHit)
            logger.LogInformation("Cache HIT for {Key}", key);

        return dto;
    }

    public async Task<List<CourseDto>> GetAllCoursesAsync(CancellationToken
ct)
    {
        var key = CacheKeys.CoursesAll;
        var dbHit = false;

        var list = await cache.GetOrCreateAsync(
            key,
            repo,
            async (state, token) =>
            {
                dbHit = true;
                logger.LogInformation("Cache MISS for {Key}  fetching from
DB", key);
                var courses = await state.GetAllAsync(token);
                return courses.Select(c => new CourseDto(
```

```
                    c.Id, c.Title, c.Code,
                    c.MaxCapacity, c.Enrollments.Count)).ToList();
            },
            tags: [CacheKeys.CoursesTag],
            cancellationToken: ct);

        if (!dbHit)
            logger.LogInformation("Cache HIT for {Key}", key);

        return list;
    }

    public async Task InvalidateCourseCacheAsync(CancellationToken ct)
    {
        logger.LogInformation("Invalidating cache tag {Tag}",
CacheKeys.CoursesTag);
        await cache.RemoveByTagAsync(CacheKeys.CoursesTag, ct);
    }
}
```

Two things to notice. First, the `state` parameter (`(repo, code)` or `repo`) is passed explicitly to the factory this avoids capturing `repo` and `code` in a closure, which would allocate a new delegate every call. In a hot path doing 1,000 req/min, those allocations add up. Second, the hit/miss log is implemented by setting a flag *inside* the factory (which only runs on miss) and reading it after `GetOrCreateAsync` returns. There is no public hit/miss API on `HybridCache` this is the recommended pattern.

Step 4 Register the service

In `Program.cs`:

```
builder.Services.AddScoped<ICachedCourseService, CachedCourseService>();
```

Step 5 Use the cached service in your handler

Update the query handler that fetches courses to use `ICachedCourseService` instead of the repository directly.

Step 6 Invalidate on writes (this is the part teams forget)

In every handler that creates, updates, or deletes a course, call `InvalidateCourseCacheAsync`. The single most common production cache bug is a write path that bypasses invalidation; the read path then serves stale data for the full L2 TTL. Make this a habit, not a “we’ll add it later”:

```
public class UpdateCourseHandler(
    ICourseRepository repo,
    ICachedCourseService cachedService)
    : IRequestHandler<UpdateCourseCommand, bool>
```

```
{
    public async Task<bool> Handle(UpdateCourseCommand command,
CancellationToken ct)
    {
        await repo.UpdateAsync(command, ct);
        await cachedService.InvalidateCourseCacheAsync(ct);
        return true;
    }
}
```

Step 7 Verify stampede protection in the logs (not just the response)

Run the API. Send 50 concurrent requests (Scalar cannot easily do this. use **PowerShell** or **`curl` in parallel** for this load spike):

```
# PowerShell
1..50 | ForEach-Object -Parallel {
    Invoke-RestMethod https://localhost:5001/api/v2/courses | Out-Null
} -ThrottleLimit 50
```

Now grep the application log:

```
# In your dotnet run output, you should see these counts
# Cache MISS for v2:courses:all  fetching from DB   (exactly 1)
# Cache HIT for v2:courses:all                       (49)
```

This is the proof. **One** miss line, **forty-nine** hit lines. If you see more than one miss line, atomic refetching is not working (most likely cause: you used `GetAsync` + manual set instead of `GetOrCreateAsync`).

Also confirm in your EF Core SQL log: exactly **one** `SELECT` against `Courses` for those 50 requests.

Step 8 Verify tag invalidation works

Issue a write that triggers invalidation, then re-run the load test:

```
curl -X PUT https://localhost:5001/api/v2/courses/1 \
  -H "Content-Type: application/json" \
  -d '{"title": "Updated title"}'
```

You should see one log line:

```
Invalidating cache tag courses
```

The next read produces a fresh MISS (good the write was visible immediately) followed by hits.

Expected result:

- Cold cache: 50 concurrent requests → 1 database query, 1 MISS log, 49 HIT logs.

- Warm cache: 50 requests → 0 database queries, 50 HIT logs.
- After invalidation: next read produces 1 MISS, then HITs again.
- Bumping `SchemaVersion` from `v2` to `v3` causes every cached entry to become unreachable on next read (no manual flush needed).

Troubleshooting

Symptom	Fix
`No service for type 'HybridCache'`	Ensure `AddHybridCache()` is called in `Program.cs`
Multiple DB queries under concurrent load	Verify you are using `GetOrCreateAsync` (not `GetAsync` + manual set). The atomic refetching only works with the factory overload
Cache never invalidates	Confirm you are calling `RemoveByTagAsync` with the same tag string used in `GetOrCreateAsync`
Hit/miss log only shows MISS	The factory body sets `dbHit = true`. Make sure the closure variable is declared in the *outer* method scope, not inside the factory
Stale data after deploy	Bump `SchemaVersion` in `CacheKeys` whenever a DTO changes shape; old entries become unreachable on the next read

Checkpoint

The TMS course endpoint now survives Monday morning traffic spikes **observably**. 50 concurrent requests produce 1 database query and a clean 1-MISS / 49-HIT log signature you can show in incident review. Tag-based invalidation works on every write path. The cache key strategy is forward-compatible with DTO changes via `SchemaVersion`. Production-readiness gap: swap the in-memory L2 for Redis (Step 1 commented snippet) when you scale beyond one pod.

Exercise 4: Tier-Aware Rate Limiting (LO 7.3)

Scenario: A misconfigured script at one training centre is sending 100 requests per second to the course search endpoint. Yesterday's outage report blamed it for 5 seconds of latency for *every* user. The fix is partitioned rate limiting: every API key gets its own bucket, paid integration partners get a higher tier, and the expensive transcript endpoint gets a separate concurrency limit (not just per-second, but "how many transcripts can run *at the same time*"). You will build this end-to-end.

Step 1 Decide what counts as "the caller"

Real APIs do not partition by IP corporate NATs put thousands of users behind one IP. Real APIs partition by **API key** (or, after M12 lands, JWT subject). For this lab, accept an `X-Api-Key` header and a tier inferred from a small lookup table:

```
// TmsApi.Api/RateLimiting/ApiKeyTier.cs
namespace TmsApi.Api.RateLimiting;

public enum ApiKeyTier { Anonymous, Free, Paid }

public static class ApiKeyResolver
{
    private static readonly Dictionary<string, ApiKeyTier> Keys =
new(StringComparer.Ordinal)
    {
        ["tms-free-demo-001"] = ApiKeyTier.Free,
        ["tms-paid-001"]      = ApiKeyTier.Paid
    };

    public static (string PartitionKey, ApiKeyTier Tier) Resolve(HttpContext
ctx)
    {
        var key = ctx.Request.Headers["X-Api-Key"].ToString();
        if (string.IsNullOrEmpty(key))
            return (ctx.Connection.RemoteIpAddress?.ToString() ??
"anonymous", ApiKeyTier.Anonymous);

        return Keys.TryGetValue(key, out var tier)
            ? (key, tier)
            : (key, ApiKeyTier.Anonymous);
    }
}
```

> In M12 you will replace this lookup with real authentication. For M7 the goal is to teach **partitioning** the mechanism is identical whether the partition key comes from a header, a JWT, or a tenant id pulled from a database.

Step 2 Configure tier-aware Token Bucket as the default policy

Open `TmsApi.Api/Program.cs`:

```
using System.Threading.RateLimiting;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.RateLimiting;
using TmsApi.Api.RateLimiting;

builder.Services.AddRateLimiter(options =>
{
    options.GlobalLimiter = PartitionedRateLimiter.Create<HttpContext,
string>(httpContext =>
    {
        var (partitionKey, tier) = ApiKeyResolver.Resolve(httpContext);

        return tier switch
        {
```

```
            ApiKeyTier.Paid => RateLimitPartition.GetTokenBucketLimiter(
                partitionKey: $"paid:{partitionKey}",
                factory: _ => new TokenBucketRateLimiterOptions
                {
                    TokenLimit = 200,
                    TokensPerPeriod = 100,
                    ReplenishmentPeriod = TimeSpan.FromSeconds(10),
                    QueueLimit = 0,
                    AutoReplenishment = true
                }),
            ApiKeyTier.Free => RateLimitPartition.GetTokenBucketLimiter(
                partitionKey: $"free:{partitionKey}",
                factory: _ => new TokenBucketRateLimiterOptions
                {
                    TokenLimit = 30,
                    TokensPerPeriod = 10,
                    ReplenishmentPeriod = TimeSpan.FromSeconds(10),
                    QueueLimit = 0,
                    AutoReplenishment = true
                }),
            _ => RateLimitPartition.GetTokenBucketLimiter(
                partitionKey: $"anon:{partitionKey}",
                factory: _ => new TokenBucketRateLimiterOptions
                {
                    TokenLimit = 10,
                    TokensPerPeriod = 5,
                    ReplenishmentPeriod = TimeSpan.FromSeconds(10),
                    QueueLimit = 0,
                    AutoReplenishment = true
                })
        };
    });

    options.RejectionStatusCode = StatusCodes.Status429TooManyRequests;
    options.OnRejected = async (context, ct) =>
    {
        var retryAfter = "10";
        if (context.Lease.TryGetMetadata(MetadataName.RetryAfter, out var
ts))
            retryAfter = ((int)ts.TotalSeconds).ToString();

        context.HttpContext.Response.Headers.RetryAfter = retryAfter;
        context.HttpContext.Response.ContentType =
"application/problem+json";
        await context.HttpContext.Response.WriteAsJsonAsync(new
ProblemDetails
        {
            Title = "Rate limit exceeded",
            Detail = $"Too many requests. Retry after {retryAfter} seconds.",
            Status = StatusCodes.Status429TooManyRequests,
```

```
                Type = "https://tms.local/errors/rate_limit_exceeded"
            }, ct);
    };
});
```

Three things every senior reviewer expects to see here, all present:

1. **Per-caller partition** heavy use by one client cannot starve another.
2. **Tier policy** paid customers get a higher ceiling without code branches in handlers.
3. **`Retry-After` from the lease metadata** not a hard-coded 10. The token-bucket limiter knows when the next token replenishes and tells the client truthfully.

Wire the middleware after `UseRouting`:

```
app.UseRateLimiter();
```

Step 2b Minimal transcript route for Session 2 (stub before Exercise 5)

Exercise 5 replaces this with the real **`202 Accepted`** + status URL + worker. For **Session 2** you still need a **real HTTP surface** so the concurrency limiter has something to measure.

Add a versioned controller action (match your API versioning setup from Session 1. For example `TranscriptsController` under **`api/v{version:apiVersion}`** or a fixed **`api/v2/transcripts`** if that is how you routed it) with:

```
[ApiController]
[Route("api/v2/transcripts")]
public class TranscriptsController : ControllerBase
{
    [HttpPost]
    [EnableRateLimiting("transcripts")]
    public IActionResult RequestTranscript([FromBody] object? _)
    {
        // Stub: Exercise 5 swaps this for enqueue + 202 + Location.
        return Ok();
    }
}
```

Adjust the route template to match your project. The **attribute `EnableRateLimiting("transcripts")`** must be present **before** you run the Step 6 burst.

Step 3 Add a concurrency limiter for the expensive endpoint

The transcript endpoint runs a 5–15 second background job per call. Token Bucket limits *how often* you can request but not *how many can run at once*. If 30 requests trigger 30 simultaneous transcript builds, the database connection pool dies. The right tool is a **concurrency limiter**, separately attached to the transcript route:

```
builder.Services.AddRateLimiter(options =>
{
    // ... GlobalLimiter from Step 2 stays as-is ...

    options.AddConcurrencyLimiter("transcripts", opt =>
    {
        opt.PermitLimit = 5;          // 5 in-flight transcripts maximum
        opt.QueueLimit = 20;          // queue up to 20 more
        opt.QueueProcessingOrder = QueueProcessingOrder.OldestFirst;
    });
});
```

Then attach the named policy to the transcript controller (it sits *on top of* the global tier policy both apply):

```
[HttpPost]
[EnableRateLimiting("transcripts")]
public async Task<IActionResult> RequestTranscript(...) { ... }
```

Step 4 Exempt health checks

A common mistake: rate limit `/health` and watch your load balancer mark the service unhealthy under load. Disable the limiter for known infrastructure endpoints:

```
app.MapHealthChecks("/health/live").DisableRateLimiting();
app.MapHealthChecks("/health/ready").DisableRateLimiting();
```

(The health check endpoints themselves are added in Exercise 9. For now, just remember the pattern.)

Step 5 Apply the search-only token policy where appropriate

For routes that need a tighter cap than the global tier (e.g., a fuzzy search that hits an external index), keep a named policy and add it explicitly:

```
options.AddTokenBucketLimiter("search", opt =>
{
    opt.TokenLimit = 10;
    opt.TokensPerPeriod = 5;
    opt.ReplenishmentPeriod = TimeSpan.FromSeconds(10);
    opt.QueueLimit = 2;
});
```

```
[HttpGet("search")]
[EnableRateLimiting("search")]
public async Task<IActionResult> SearchCourses(
    [FromQuery] string? term, CancellationToken ct)
{
    var results = await mediator.Send(new SearchCoursesQuery(term), ct);
    return Ok(results);
}
```

Step 6 Test all three layers

Run the API. Test the **anonymous** tier first (no header):

```
1..15 | ForEach-Object {
    $r = Invoke-WebRequest https://localhost:5001/api/v2/courses
-SkipHttpErrorCheck
    "$_ : $($r.StatusCode) Retry-After=$($r.Headers.'Retry-After')"
}
```

Now compare the **paid** tier:

```
1..15 | ForEach-Object {
    $r = Invoke-WebRequest https://localhost:5001/api/v2/courses `
        -Headers @{ "X-Api-Key" = "tms-paid-001" } -SkipHttpErrorCheck
    "$_ : $($r.StatusCode)"
}
```

Test the concurrency limiter on **`POST /api/v2/transcripts`** (the Session 2 stub from Step 2b; Exercise 5 upgrades the handler, **`EnableRateLimiting("transcripts")`** stays):

```
1..30 | ForEach-Object -Parallel {
    Invoke-WebRequest https://localhost:5001/api/v2/transcripts `
        -Method POST -Body '{"studentId":1}' -ContentType 'application/json'
`
        -SkipHttpErrorCheck | Out-Null
} -ThrottleLimit 30
```

Expected result:

- Anonymous: requests 1–10 succeed (status 200), 11+ return `429` with `Retry-After: ~10`.
- Paid: all 15 succeed the paid tier has TokenLimit=200.
- Transcripts: at most 5 in flight at any moment (visible in worker logs), the next 20 queue, and the 26th onwards return `429`.
- The `Retry-After` value in 429 responses reflects when the next token will be available, not a hard-coded constant.

Troubleshooting

Symptom	Fix
Rate limiting not applied	Ensure `app.UseRateLimiter()` runs *after* `UseRouting()` and *before* `MapControllers()`
Paid tier still gets 429 quickly	Confirm `X-Api-Key` header value matches a key in `ApiKeyResolver.Keys` exactly (case-sensitive)
`Retry-After` header always says `10`	The `OnRejected` handler must call `Lease.TryGetMetadata(MetadataName.RetryAfter, ...)` using a static value defeats the limiter

Symptom	Fix
Concurrency limiter never queues	Confirm `[EnableRateLimiting("transcripts")]` attribute is present on the action and `AddConcurrencyLimiter("transcripts", ...)` is registered
`/health/live` returns 429 under load	Add `.DisableRateLimiting()` to the health-check `MapHealthChecks(...)` call

Checkpoint

Session Integration Challenge: Cache + limit, working together

When both exercises are done, run this sequence and confirm every layer cooperates:

```
# Empty the cache by restarting the API once.
# Then:

# 1. Anonymous tier hits the rate limit before the cache warms—bad UX, by
design
1..15 | ForEach-Object {
    $r = Invoke-WebRequest https://localhost:5001/api/v2/courses
-SkipHttpErrorCheck
    "anon $_ : $($r.StatusCode)"
}

# 2. Paid tier sails through and warms the cache for everyone
1..15 | ForEach-Object {
    $r = Invoke-WebRequest https://localhost:5001/api/v2/courses \
        -Headers @{ "X-Api-Key" = "tms-paid-001" } -SkipHttpErrorCheck
    "paid $_ : $($r.StatusCode)"
}

# 3. Anonymous tier comes back. Cache is now warm—only retries hit, no DB
load.
1..15 | ForEach-Object {
    $r = Invoke-WebRequest https://localhost:5001/api/v2/courses
-SkipHttpErrorCheck
    "anon-warm $_ : $($r.StatusCode)"
}
```

Open the application log (or the dotnet run terminal) and grep the `Cache HIT` / `Cache MISS` lines. You should see:

- A burst of MISS at the start while the paid tier warms the cache.
- A long run of HIT for the warm anonymous run.
- Exactly **one** EF Core SQL line for the entire `Courses` query, the rest came from cache.

That is the proof of “cache is solving real load,” not a synthetic 50-parallel-curl test.

Session 2 Checkpoint

Tick all of these before you leave the room:

- ☐ 50 concurrent requests on a cold cache produce **1** `Cache MISS` line and **49** `Cache HIT` lines, **and** exactly one EF Core `SELECT` against `Courses`.
- ☐ An anonymous client gets 200 OKs for the first 10 requests in any 10 s window, then `429 Too Many Requests` with `application/problem+json` body.
- ☐ The **same calls** with `X-Api-Key: tms-paid-001` succeed, the paid tier is not throttled at the same rate.
- ☐ The 429 response carries a `Retry-After` value derived from the lease metadata, not a hard-coded constant.
- ☐ A burst of 30 transcript POSTs respects the **transcripts** concurrency limiter (at most **5** in flight; queue then 429s—see Exercise 4 · Step 6 in this handout). After Exercise 5, confirm the same cap in worker logs, not only HTTP status codes.
- ☐ After a **course write** that triggers invalidation (for example **`PUT /api/v2/courses/{id}`** with the `id` your V2 controller uses, typically **`int`** per M6), the next **`GET`** for the affected catalogue/detail produces a fresh `Cache MISS` and the response reflects the change immediately.
- ☐ `dotnet test` is still green.

Common Errors: Session 2

Symptom	Likely cause	Quick fix
Multiple `Cache MISS` lines under concurrent load	Used `GetAsync` + manual `SetAsync`	Switch to `GetOrCreateAsync(key, state, factory, options, tags, ct)`—the atomic refetch only exists in the factory overload
Cache invalidation does nothing	Tag string mismatch between `GetOrCreateAsync(... tags: [...])` and `RemoveByTagAsync(...)`	Centralise the tag in `CacheKeys.CoursesTag` and reference it from both call sites
Stale shape after deploy	DTO changed but `SchemaVersion` was not bumped	Bump `SchemaVersion` in `CacheKeys` whenever a DTO field is added, removed, or renamed
`Retry-After: 10` never changes	`OnRejected` returns a hard-coded string	Read `Lease.TryGetMetadata(MetadataNam`

Symptom	Likely cause	Quick fix
		`e.RetryAfter, out var ts)` and stringify `(int)ts.TotalSeconds`
Anonymous tier never throttled	`GlobalLimiter` not registered, or `app.UseRateLimiter()` missing	Register `PartitionedRateLimiter.Create<HttpContext, string>(...)` in `AddRateLimiter` and call `app.UseRateLimiter()` after `UseRouting()`
Concurrency limiter never queues	`[EnableRateLimiting("transcripts")]` attribute missing on the action	Add the attribute, and confirm `AddConcurrencyLimiter("transcripts", ...)` is registered

Bridge to Session 3

The cache stops the database from melting. The rate limiter stops one bad client from affecting others. Both improve the dashboard's *steady state*. Tomorrow's session adds the two patterns the dashboard needs for *long-running and real-time* work: a transcript that takes 15 seconds without blocking the request thread (Exercise 5) and a notification that arrives the moment the transcript is ready (Exercise 6). Skim the **Lab Session 3** handout before the lecture; pre-reading saves about an hour at the keyboard.